An Allocator-aware `inplace_vector`

Document #:	P3160R0
Date:	2024-02-15 14:04 EST
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Pablo Halpern <phalpern@halpernwightsoftware.com>

Contents

§ 1 Abstract

The `inplace_vector` proposal, [P0843R10] is moving forward without allocator support. This paper proposes that `inplace_vector` *should* have allocator support ane explores the pro and con of adding such support direction into `inplace_vector` vs. a separate `basic_inplace_vetor` class template. This proposal is separate from [P0843R10] so that the latter can move forward more quickly, while allocator-specific policies are still being worked out.

§ 2 Motivation

§ 2.1 General Motivation for allocator-aware types

Note: The text below is borrowed nearly verbetim from [P3002R1], which proposes a general policy for when types should use allocators

Memory management is a major part of building software. Numerous facilities in the C++ Standard library exist to give the programmer maximum control over how their program uses memory:

— `std::unique_ptr` and `std::shared_ptr` are parameterized with *deleter* objects that control, among other things, how memory resources are reclaimed.

— `std::vector` is preferred over other containers in many cases because its use of contiguous memory provides optimal cache locality and minimizes allocate/deallocate operations. Indeed, the LEWG has spent a lot of time on `flat_set` ([P1222R0]) and `flat_map` ([P0429R3]), whose underlying structure defaults to `vector` for this reason.

— Operators `new` and `delete` are replaceable, giving programmers global control over how memory is allocated.

— The C++ object model makes a clear distinction between an object's memory footprint and it's lifetime.

— Language constructs such as `void*` and `reinterpet_cast` provide fine-grained access to objects' underlying memory.

— Standard containers and strings are parameterized with allocators, providing object-level control over memory allocation and element construction.

This fine-grained control over memory that C++ gives the programmer is a large part of why C++ is applicable to so many domains — from embedded systems with limited memory budgets to games, high-frequency trading, and scientific simulations that require cache locality, thread affinity, and other memory-related performance optimizations.

An in-depth description of the value proposition for allocator-aware software can be found in [P2035R0]. Standard containers are the most ubiquitous examples of *allocator-aware* types. Their `allocator_type` and `get_allocator` members and allocator-parameterized constructors allow them to be used like Lego® parts that can be combined and nested as necessary while retaining full programmer control over how the whole assembly allocates memory. For *scoped allocators* — those that apply not only to the top-level container, but also to its elements — having each element of a container support a predictable allocator-aware interface is crucial to giving the programmer the ability to allocate all memory from a single memory resource, such as an arena or pool. Note that the allocator is a *configuration* parameter of an object and does not contribute to its value.

In short, the principles underlying this policy proposal are:

1. **The Standard Library should be general and flexible**. To the extent possible, the user of a library class should have control over how memory is allocated.

2. **The Standard Library should be consistent**. The use of allocators should be consistent with the existing allocator-aware classes and class templates, especially those in the containers library.

3. **The parts of the Standard Library should work together**. If one part of the library gives the user control over memory allocation but another part does not, then the second part undermines the utility of the first.

4. **The Standard Library should encapsulate complexity**. Fully general application of allocators is potentially complex and is best left to the experts implementing the Standard Library. Users can choose their own subset of desirable allocator behavior only if the underlying Library classes allow them to choose their preferred approach, whether it be stateless allocators, statically typed allocators, polymorphic allocators, or no allocators.

§ 2.2 Motivation for an Allocator-aware `inplace_vector`

Although the objects stored in an `std::inplace_vector`, as proposed in [P0843R10] can be initialized with any set of valid constructor arguments, including allocator arguments, the fact that the `inplace_vector` itself is not allocator-aware prevents it from working consistently with other parts of the standard library, specifically those parts that depend on *uses-allocator construction* (section [allocator.uses.construction]) in the standard). For example:

```
pmr::monotonic_buffer_resource rsrc;
pmr::polymorphic_allocator<> alloc{ &rsrc };
using V = inplace_vector<pmr::string, 10>;
V v = make_obj_using_allocator<V>(alloc, { "hello", "goodbye" });
assert(v[0].get_allocator() == alloc);  // FAILS
```

Even though an allocator is supplied, it is not used to construct the `pmr::string` objects within the resulting `inplace_vector` object because `inplace_vector` does not have the necessary hooks for `make_obj_using_allocator` to recognize it as being allocator-aware. Note that, although this example and the ones that follow use `pmr::polymorphic_allocator`, the same issues would apply to any scoped allocator.

Uses-allocator construction is rarely used directly in user code. Instead, it is used within the implementation of standard containers and scoped allocators to ensure that the allocator used to construct the container is also used to construct its elements. Continuing the example above, consider what happens if an `inplace_vector` is stored in a `pmr::vector`, compared to storing a truly allocator-aware type (`pmr::string`):

```
pmr::vector<pmr::string> vs(alloc);
pmr::vector<V>           vo(alloc);

vs.emplace_back("hello");
vo.emplace_back({ "hello" });

assert(vs.back().get_allocator() == alloc);      // OK
assert(vo.back()[0]->get_allocator() == alloc);  // FAILS
```

An important invariant when using a scoped allocator such as `pmr::polymorphic_allocator` is that the same allocator is used throughout an object hierarchy. It is impossible to ensure that this invariant is preserved when using `std::inplace_vector`, even if each element is originally inserted with the correct allocator, because `inplace_vector` does not remember the allocator used to construct it and cannot therefore supply the allocator to new elements.

§ 3 Design options

There are several possible designs for an allocator-aware `inplace_vector`:

§ 3.1 `inplace_vector<class T, size_t N, class Alloc = std::allocator<T>>`

The allocator would be stored in the object and returned by get_allocator(). If `uses_allocator_v<T, Alloc>` is true, then elements are constructed via *uses-allocator construction* with the supplied allocator. For `std::allocator`, no space would be need to be taken up – in fact, that should be a requirement, so that `sizeof(inplace_vector<T, N>)` is guaranteed to be *N* * `sizeof(T)`, i.e., no size overhead for the default allocator case.

Pros: Simplest design, most consistent with other containers, zero runtime overhead for the default case.

Cons: If `uses_allocator_v<T, Alloc>` is false and `Alloc` is a non-empty class, then space is wasted storing an unused allocator. Alternative a) We could create a special case that a default-constructed Alloc could be returned from `get_allocator()`, if `Alloc` is default-constructible. Alternative b) In this case, inplace_vector could be non-allocator-aware, `allocator_type` and `get_allocator()` would not be defined and no constructors would accept an allocator argument. Alternative c) Supplying an allocator other than `std::allocator` to for a type that cannot use it could be ill-formed; unfortunately, this would make generic programming somewhat more difficult.

§ 3.2 `inplace_vector<class T, size_t N>`

If `T::allocator_type` exists, then `inplace_vector<class T, size_t N>::allocator_type` would also exist, as would `get_allocator()` and allocator-accepting constructors. Otherwise, the instantiation would not be allocator aware.

Pros: Easiest for users – no need to consider whether to supply an allocator template argument.

Cons: If `T` is allocator-aware but the user doesn't want to take advantage of that, an allocator is stored unnecessarily. Worse, there are no obvious work-arounds to avoid storing and using an allocator. Also, this approach is inconsistent with other containers.

§ 3.3 `inplace_vector<class T, size_t N, class Alloc = *see below* >`

The *see below* type is `T::allocator_type` if such a type exists, and `std::allocator<byte>` otherwise. This approach combines the advantages of the previous 2. Unlike approach 2, however, the user can explicitly specify an allocator (presumably `std::allocator`) other than `T::allocator_type`, if they don't want to take up space in the vector for an allocator that will just be defaulted anyway.

§ 3.4 `basic_inplace_vector<class T, size_t N, class Alloc>`

This approach has a separate template, `basic_inplace_vector`, for allocator-aware inplace vector.

Pros: Reduces complications with `inplace_vector` specification and implementation. Potentially reduces compile time compared to some of the preceding options.

Cons: The two types of inplace vector are not compatible. The user needs to make a decision, especially in generic code, whether they ever expect to have allocator-aware elements. It is easy to choose the shorter name at the expense of breaking scoped allocation in generic code.

§ 4 Compile time data

Compile time experiments with a subset of an `inplace_vector` implementation compare the status quo to option 3.1, described above. The results below show that there is no appreciable penalty for non-allocator-aware element types (e.g., `inplace_vector<int, N>`) but a noticeable impact for allocator-aware elements (e.g., `inplace_vector<string, N>`). Further experimentation is needed to see if compile-times can be reduced. The experiments can be found at https://github.com/phalpern/WG21-halpern/tree/main/P3160-AA-inplace_vector/

test name	compiler	status quo	option 3.1	increase
Non-AA T	g++	3.99	3.94	-1%
AA T	g++	4.39	5.92	35%
Blended	g++	6.47	8.14	26%

test name	compiler	status quo	option 3.1	increase
Non-AA T	clang++	4.39	4.38	0%
AA T	clang++	4.99	7.2	44%
Blended	clang++	7.51	9.93	32%

§ 5 Conclusion

The standard should remain consistent with respect to allocator-aware containers. There are several approaches possible for making `inplace_vector` conform and work with scoped allocators. WG21 should adopt one of them, ideally in the same standard as `inplace_vector`.

§ 6 Wording

No wording yet. It depends on the direction that LEWG chooses to take.

§ 7 References

[P0429R3] Zach Laine. 2016-08-31. A Standard flat_map.
https://wg21.link/p0429r3

[P0843R10] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. `inplace_vector`.
http://wg21.link/P0843R10

[P1222R0] Zach Laine. 2018-10-02. A Standard flat_set.
https://wg21.link/p1222r0

[P2035R0] Pablo Halpern, John Lakos. 2020-01-13. Value Proposition: Allocator-Aware (AA) Software.
https://wg21.link/p2035r0

[P3002R1] Pablo Halpern. Policies for Using Allocators in New Library Classes.
http://wg21.link/P3002R1